

Lab 1-1: Basic Frameworks

<삼성 AI 전문가 교육과정> 실습

서울대학교 바이오지능 연구실 (장병탁 교수)

최우석, 정성준

2025.05.27

Biointelligence Laboratory

Dept. of Computer Science and Engineering

Seoul National University



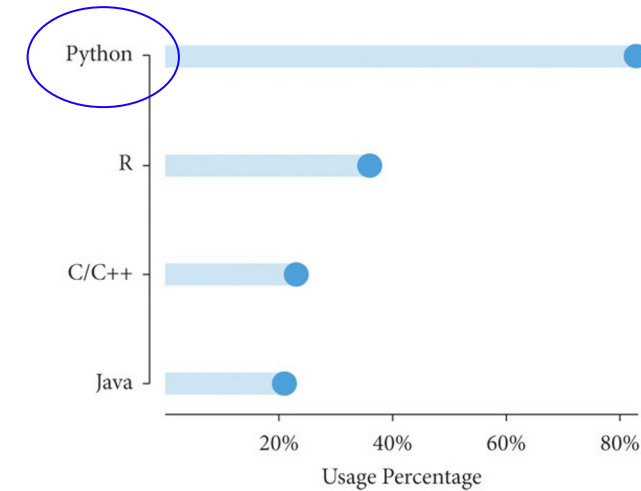
Contents

- Coding Environment Setting
- Numpy
- Pytorch basics

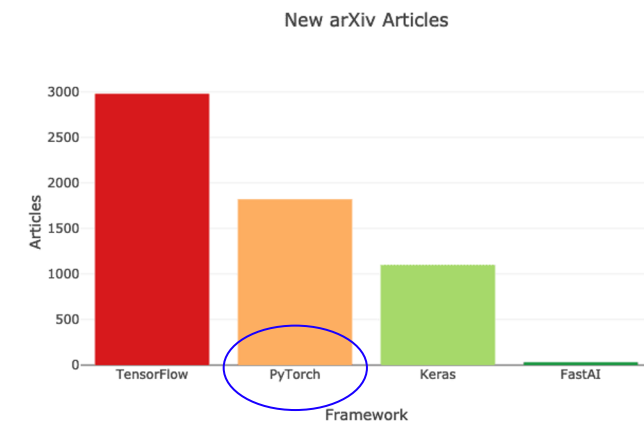
Programming language for Machine Learning



- ML 연구 분야에 있어서 대체하기 어려운 프로그래밍 언어
- Deep Learning Frameworks
 - Tensorflow, Pytorch, Keras, ...
- Python Libraries
 - Numpy, Jupyter Notebook, Matplotlib, Pandas, ...

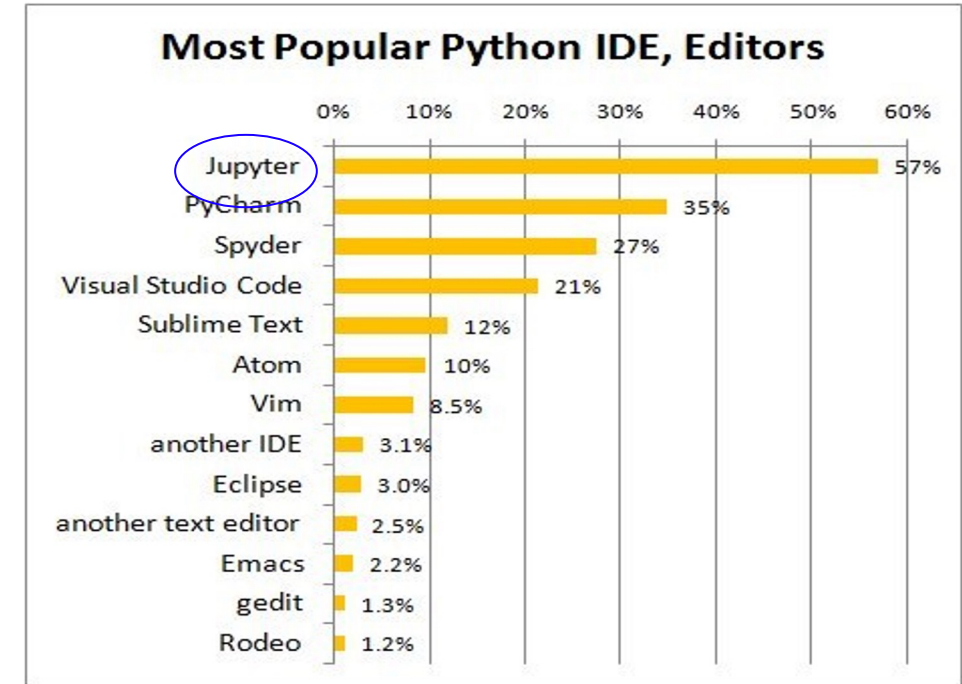
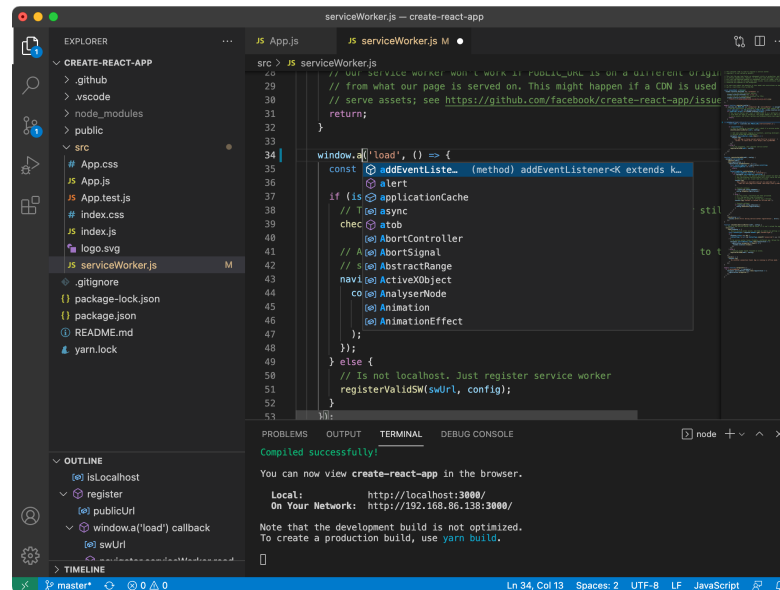


A Fortran-Keras Deep Learning Bridge for Scientific Computing - Scientific Figure on ResearchGate. Available from: https://www.researchgate.net/figure/a-Usage-of-programming-languages-for-machine-learning-and-data-science-Statistics-are_fig1_343965443 [accessed 1 Jun, 2023]



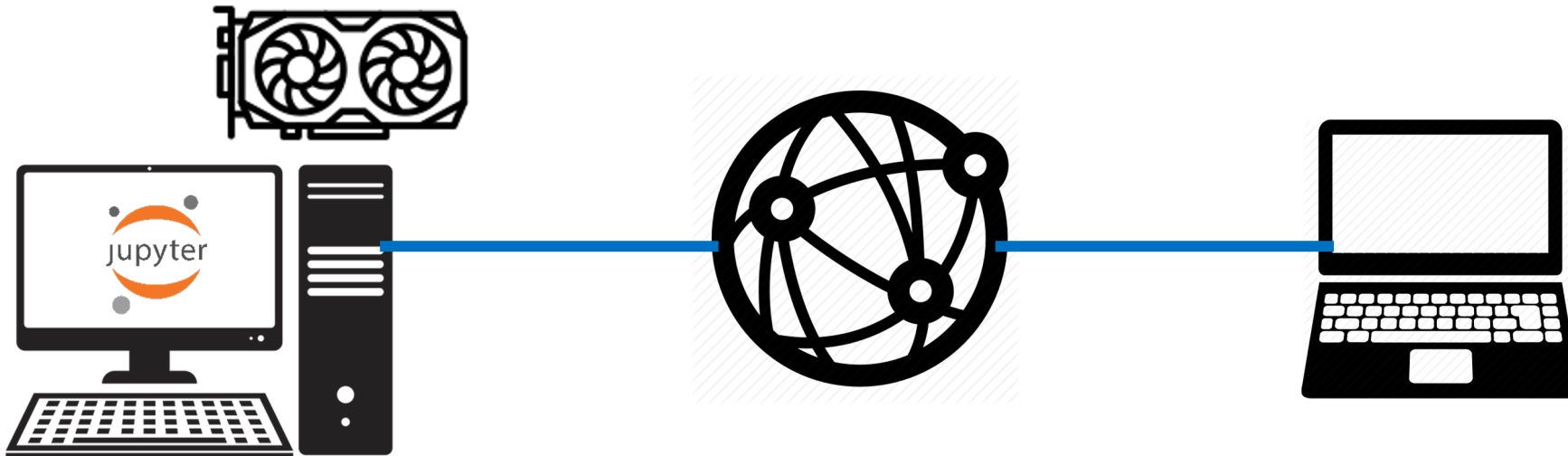
Python IDE (Integrated Development Environment)

- Python Code Editors
 - 편집기
 - Vscode, vim, 메모장, ...
- Python IDE
 - Code Editor + Debugging + Compilation tools + ...
 - Jupyter, PyCharm, Spyder, Vscode, ...



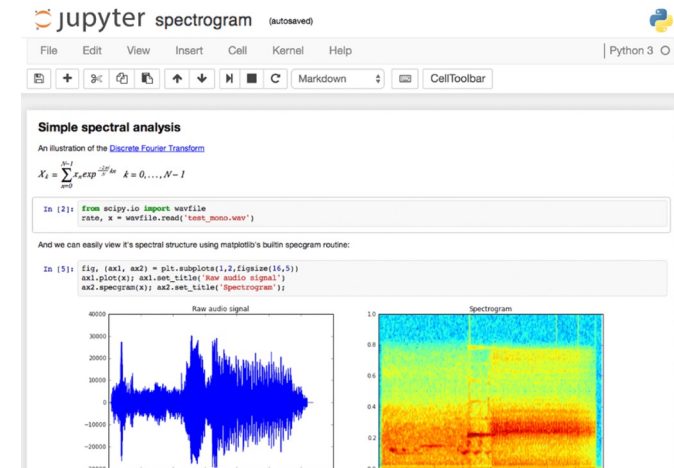
Jupyter Notebook

- 웹 브라우저에서 파이썬 코드를 작성하고 실행해 볼 수 있는 개발 도구
 - 원격 코딩 가능
 - 코드 블록 단위로 실행 / 디버깅
 - Text block을 이용한 문서화
 - Figure plotting 등 GUI



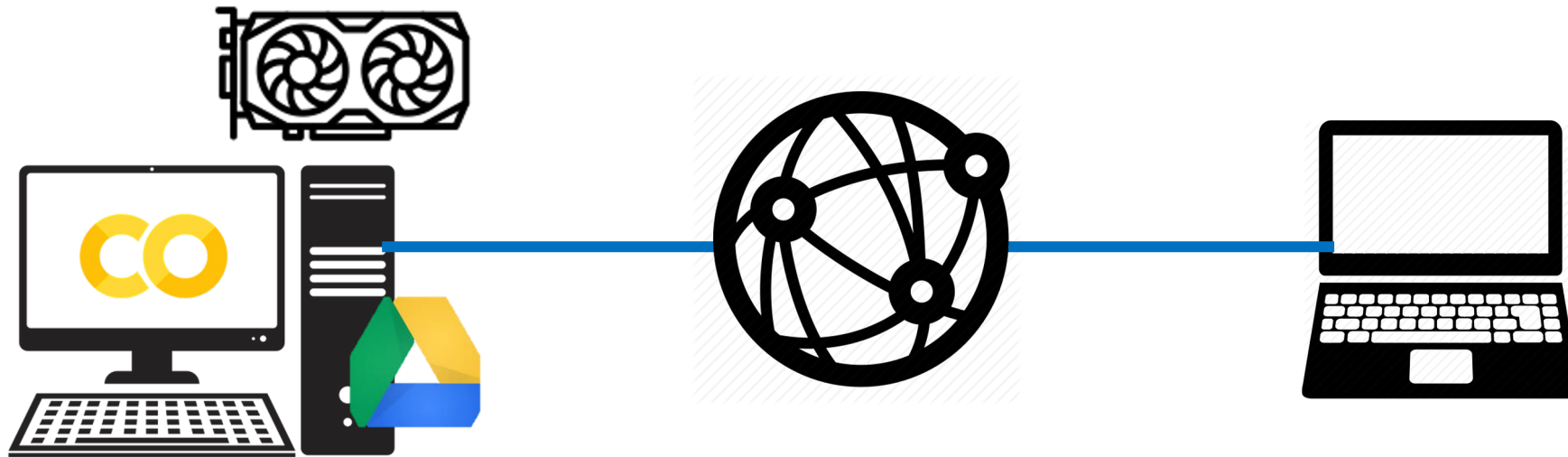
147.46.123.123

147.46.123.123:8888

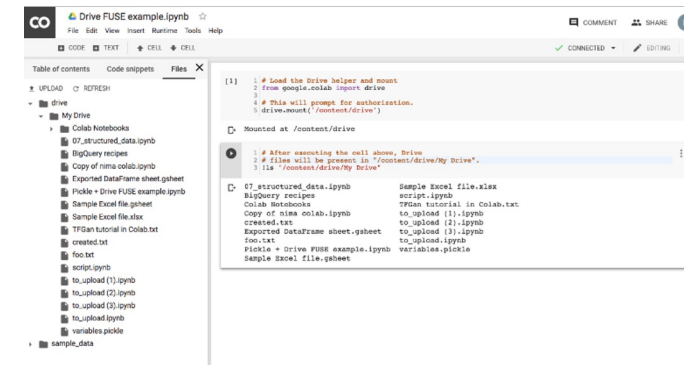


Google Colab

- **Google Colaboratory = Google Drive + Jupyter Notebook**
 - 구글 계정 전용의 가상 머신 지원 – **GPU 포함**
 - Google drive 문서와 같이 링크만으로 접근 / 협업 가능
 - 필요한 여러 패키지가 미리 설치되어 있음
 - 약간의 딜레이 존재



Google server



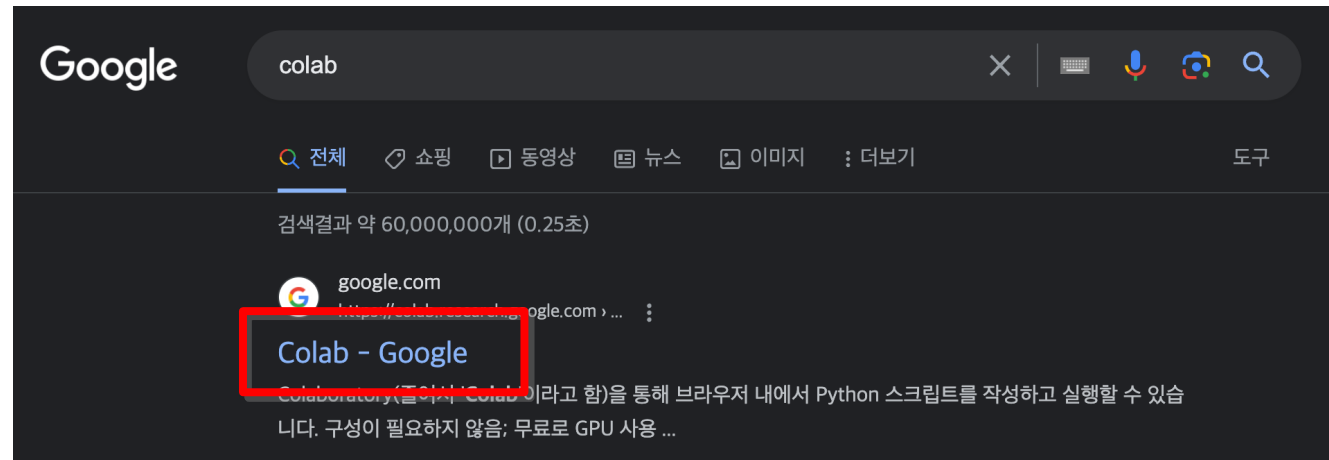
Google Colab

■ 사용법

- 개인 구글 계정으로 접속
- 구글에 Colab 검색

(<https://colab.research.google.com> 접속)

- 파일 -> 새노트 선택



Google Colab

■ 파일 이름 변경

The screenshot shows the Google Colab interface. At the top, the file name 'Lab1_1.ipynb' is highlighted with a red box, and a yellow box labeled '이름 변경' (Rename) is next to it. Below the file name, there are buttons for '+ 코드' (Code) and '+ 텍스트' (Text), both highlighted with red boxes, and a yellow box labeled '셀 추가' (Add cell) is next to them. The main area contains two cells: a 'Text Cell!' and a 'Code cell'. The 'Text Cell!' contains a list of instructions about markdown and docstring, and the 'Code cell' contains a Python code snippet. To the right of each cell is a yellow box labeled 'Text cell' and 'Code cell' respectively. At the bottom, there is a toolbar with '+ 코드' and '+ 텍스트' buttons, both highlighted with red boxes, and a yellow box labeled '셀 추가' (Add cell) is next to them.

■ Code cell, Text cell

- .ipynb 파일은 code cell과 text cell로 구성
- 각 셀 하단에 마우스를 대거나, 화면 좌상단 버튼으로 셀 추가 가능
- 셀 선택(마우스) 후 셀 우상단 삭제버튼으로 셀 삭제 가능

Google Colab

■ Code cell

- 일반적인 파이썬 코딩 방식과 동일
- 각 셀은 한번에 실행할 단위를 뜻함
- 실행 이후에도 메모리는 유지되어 다른 셀 실행 시 영향을 줌
 - 런타임 다시 시작 시 초기화
- 상단 메뉴의 런타임
 - 실행 중인 셀 중단
 - 런타임 다시 시작

The screenshot shows a code cell in Google Colab. A yellow box labeled '실행 번호' (Execution Number) points to the cell identifier '[1]'. Another yellow box labeled '실행 (Ctrl + Enter)' (Run) points to the play button icon. A third yellow box labeled '실행 결과' (Execution Result) points to the output area showing the number '3'. The code in the cell is as follows:

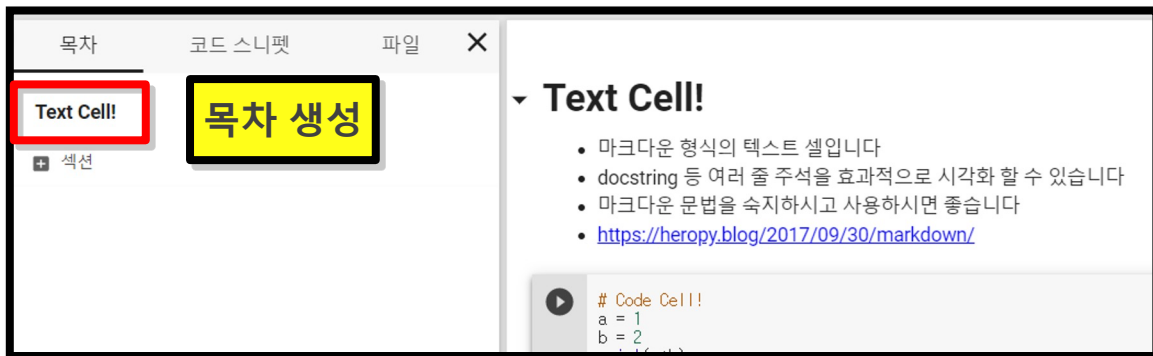
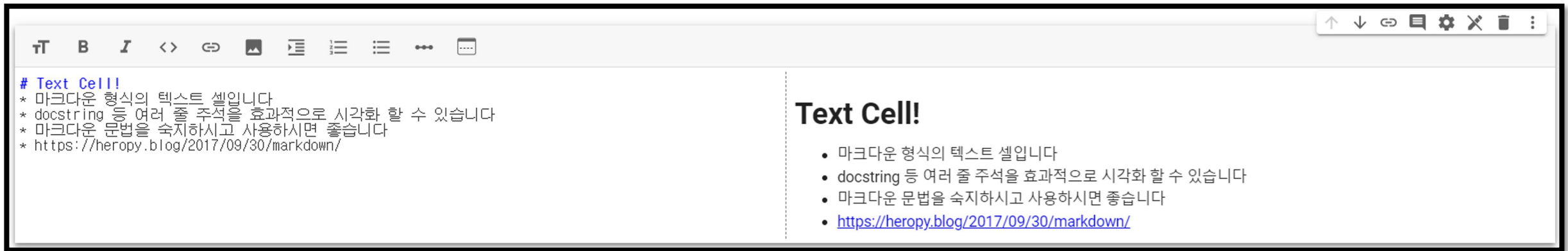
```
# Code Cell!  
a = 1  
b = 2  
print(a+b)  
  
# Ctrl+Enter 로 해당 코드 셀 실행
```

The screenshot shows the 'Run' menu in Google Colab. A yellow box labeled '실행 중단 런타임 재시작' (Stop execution and restart runtime) points to the '실행 중단' (Stop execution) and '런타임 다시 시작...' (Restart runtime...) options. The menu also shows other options like '모두 실행' (Run all), '이전 셀 실행' (Run previous cell), and '초점이 맞춰진 셀 실행' (Run selected cell).

Google Colab

■ Text cell

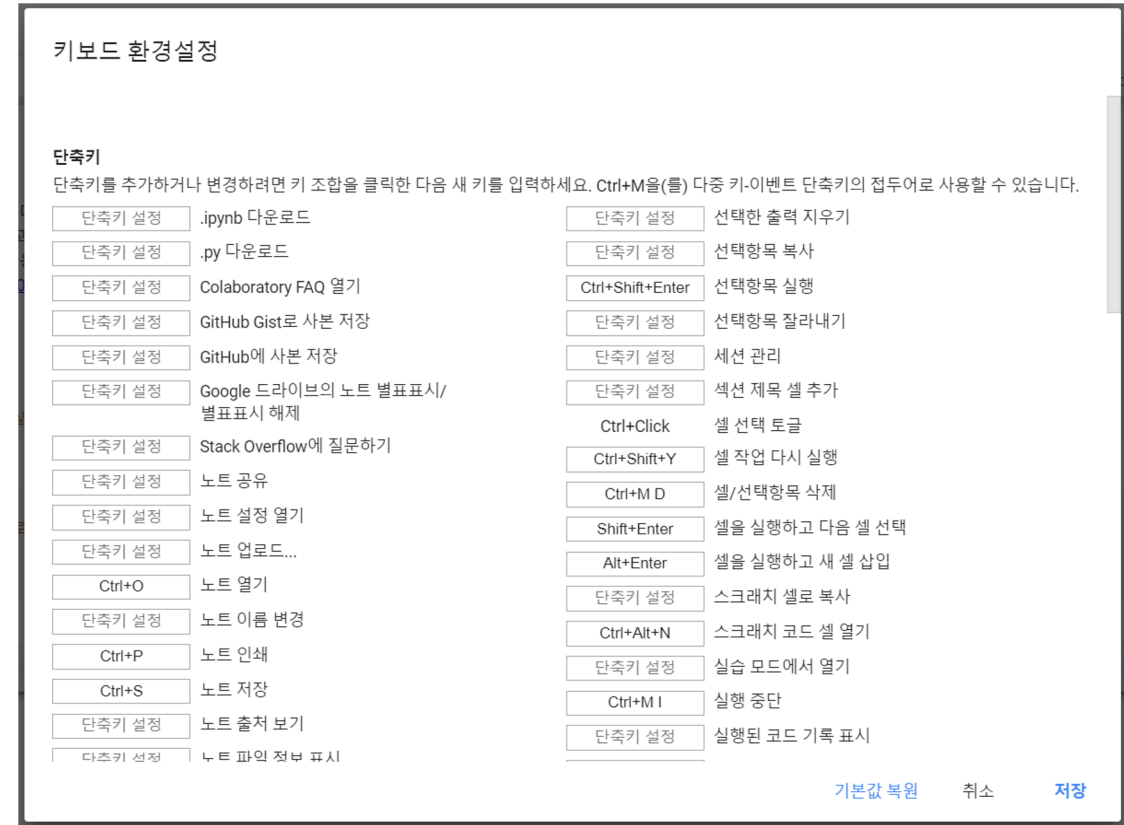
- 여러 줄 주석의 효과적인 시각화
- 마크다운(Markdown) 문법
- 자동 목차 생성



Google Colab

■ 단축키

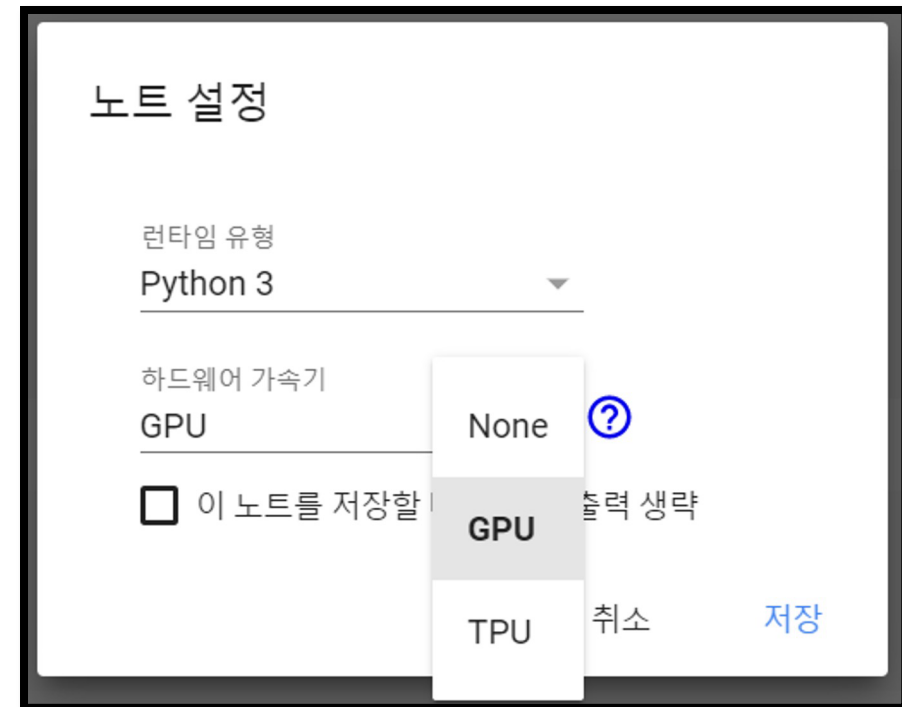
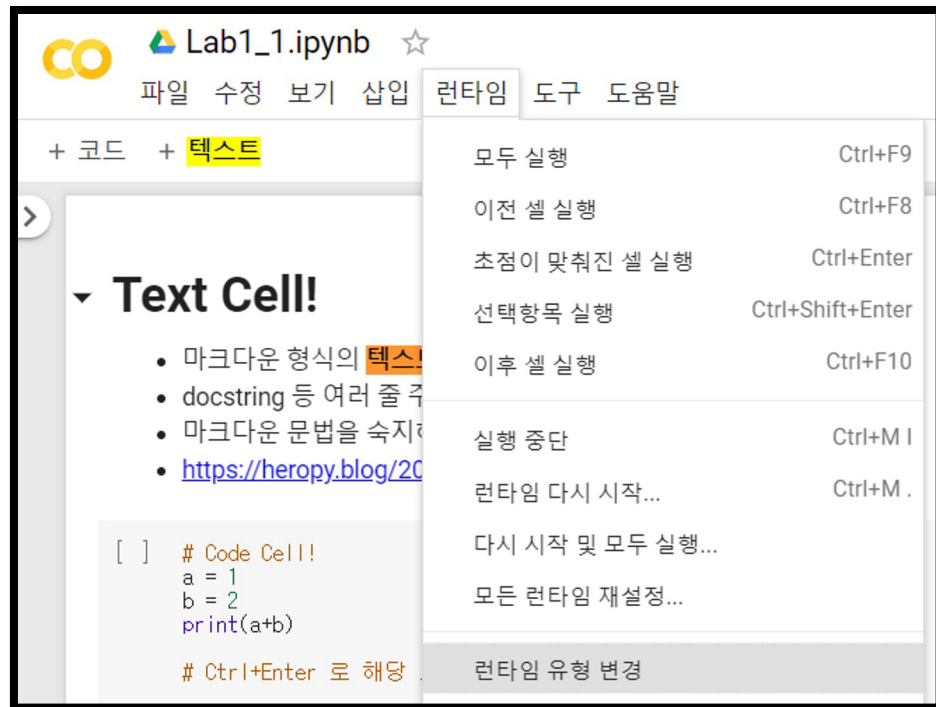
- 대부분의 작업은 단축키로 실행 가능
- 단축키 설정 가능
- 단축키 설정화면 – Ctrl+M H
- 유용한 단축키
 - 코드 셀 생성 – Ctrl+M A(B)
 - 코드 셀 실행 – Ctrl+Enter
 - 셀 삭제 – Ctrl+M D
 - 실행중인 셀 중단 – Ctrl+M I
 - 런타임 다시 시작 – Ctrl+M .
 - 코드(텍스트) 셀로 변환 – Ctrl+M Y(M)
 - 마지막 셀 작업 실행취소 – Ctrl+Shift+Z



Google Colab

■ GPU 설정

- 런타임 -> 런타임 유형 변경 -> 하드웨어 가속기를 GPU로 변경



Google Colab

■ 명령어 실행하기

- 코드 셀에 !를 붙이고 터미널 명령어를 입력하여 실행하면 터미널에서 실행하는 것과 같은 결과가 출력됨
- 예외로 cd 명령어는 %cd '/your/desired/path'

```
▶ # !cat /etc/issue.net # OS
# !cat /proc/cpuinfo # CPU
# !cat /proc/meminfo # Memory
# !df -h # Disk
!nvidia-smi # GPU
```

Mon Sep 16 07:49:42 2019

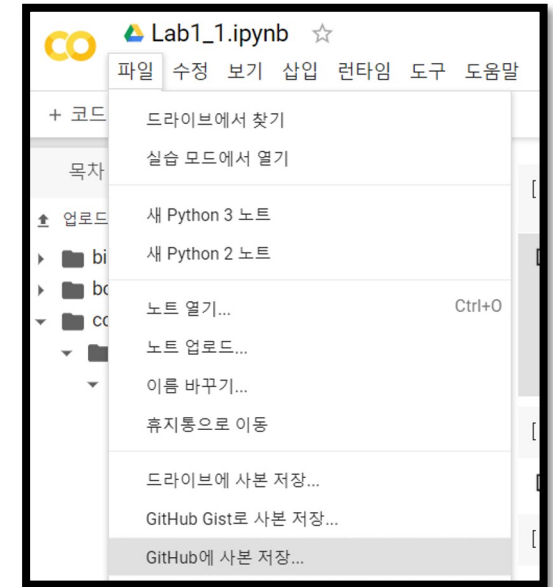
NVIDIA-SMI 430.40				Driver Version: 418.67				CUDA Version: 10.1			
GPU	Name	Persistence-M	Bus-Id	Disp.A	Volatile Uncorr. ECC						
Fan	Temp	Perf	Pwr:Usage/Cap	Memory-Usage	GPU-Util	Compute M.					
0	Tesla K80	Off	00000000:00:04.0 Off			0					
N/A	52C	P8	32W / 149W	0MiB / 11441MiB	0%	Default					

Processes:				GPU Memory
GPU	PID	Type	Process name	Usage
No running processes found				

Google Colab

■ Github 연동

- 단일 .ipynb 파일을 clone 하는 방법
 - <https://colab.research.google.com/github/>
 - 파일 -> 드라이브에 사본 저장
- 전체 repository cloning
 - !git clone project.git
- github repository에 파일을 올리는 방법
 - 파일 - Github에 사본 저장 선택
 - 저장소, 브랜치, 경로 지정



GitHub으로 복사

저장소: 브랜치:

파일 경로
Lab1_1.ipynb

변경사항 설명 메시지
Colaboratory를 통해 생성됨

☒ Colaboratory 링크 추가

취소 확인

Google Colab

- Colab에 파일 가져오기
 - 로컬에서 파일 가져오기
 - 구글 드라이브 연동
 - 간단한 인증 절차 이후 구글 드라이브의 파일을 Colab에서 접근 가능

Google Colab - tips

■ 장시간 사용할 때의 issue들

- The '**maximum lifetime**' of a running notebook is **12 hours** (browser open)
- An '**Idle**' notebook instance cuts-off after **90 minutes**
- You can have a maximum of **2 notebooks** running concurrently
- If you close the notebook window and open it while the instance is still running, the cell outputs and variables will still persist. However if the notebook instance has been recycled, your cell outputs and variables will no longer be available.

- GPU 사용/미사용 관계없이 최대 12시간
- 아무것도 안 하는 idle 상태 돌입 이후 90분에 런타임 자동으로 shutdown
- 동시에 최대 2개의 notebook 가동 가능



1-2

Numpy, Pytorch

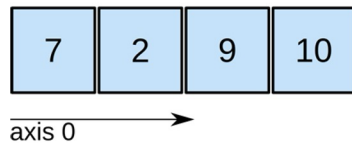
Numpy

- Numpy

- Numpy는 multi-dimensional array 연산을 편리하게 하기 위한 라이브러리
- Numpy 모듈 설치 - *pip install numpy*
 - Colab에는 기본적으로 설치되어 있다.
- Import - *import numpy (as np)*

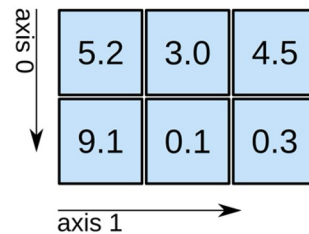
Numpy array

1D array



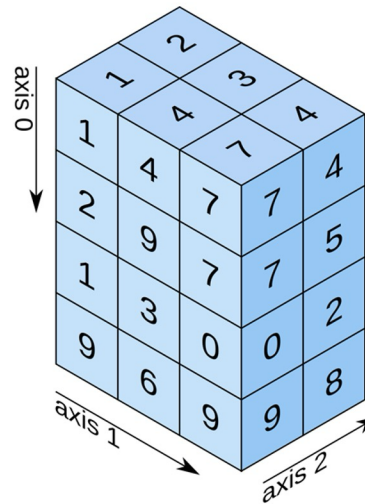
shape: (4,)

2D array



shape: (2, 3)

3D array



shape: (4, 3, 2)

출처:

<https://www.safaribooksonline.com/library/view/elementary-scipy/9781491922927/ch01.html>

- Numpy array는 모양(shape)과 데이터 타입(dtype)을 가진다.

Tensor

■ Tensor?

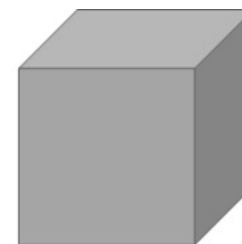
- Multi-dimension array (vector, matrix, ...)
- 특별한 케이스가 아니라면, 딥러닝 프레임워크는
 - tensor 형태의 입력과
 - tensor 형태의 모델 가중치를 적절히 연산하여
 - tensor 형태의 출력을 내는 과정을 반복함
- Tensor 형태의 데이터를 잘 다룰 수 있어야 함



vector



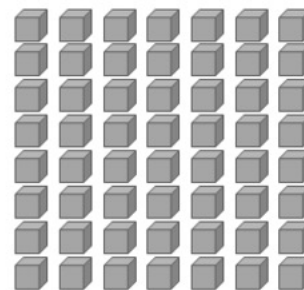
matrix



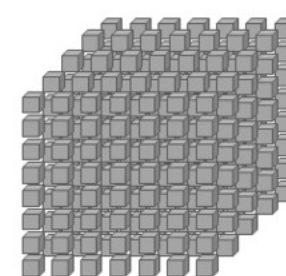
3d-tensor



4d-tensor



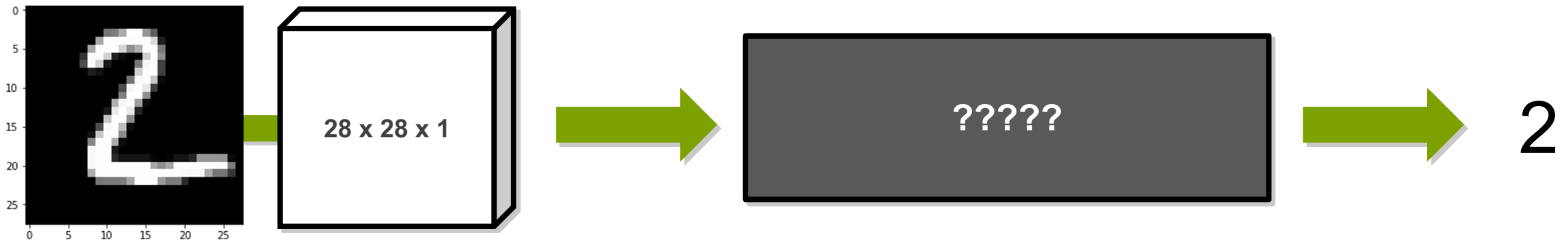
5d-tensor



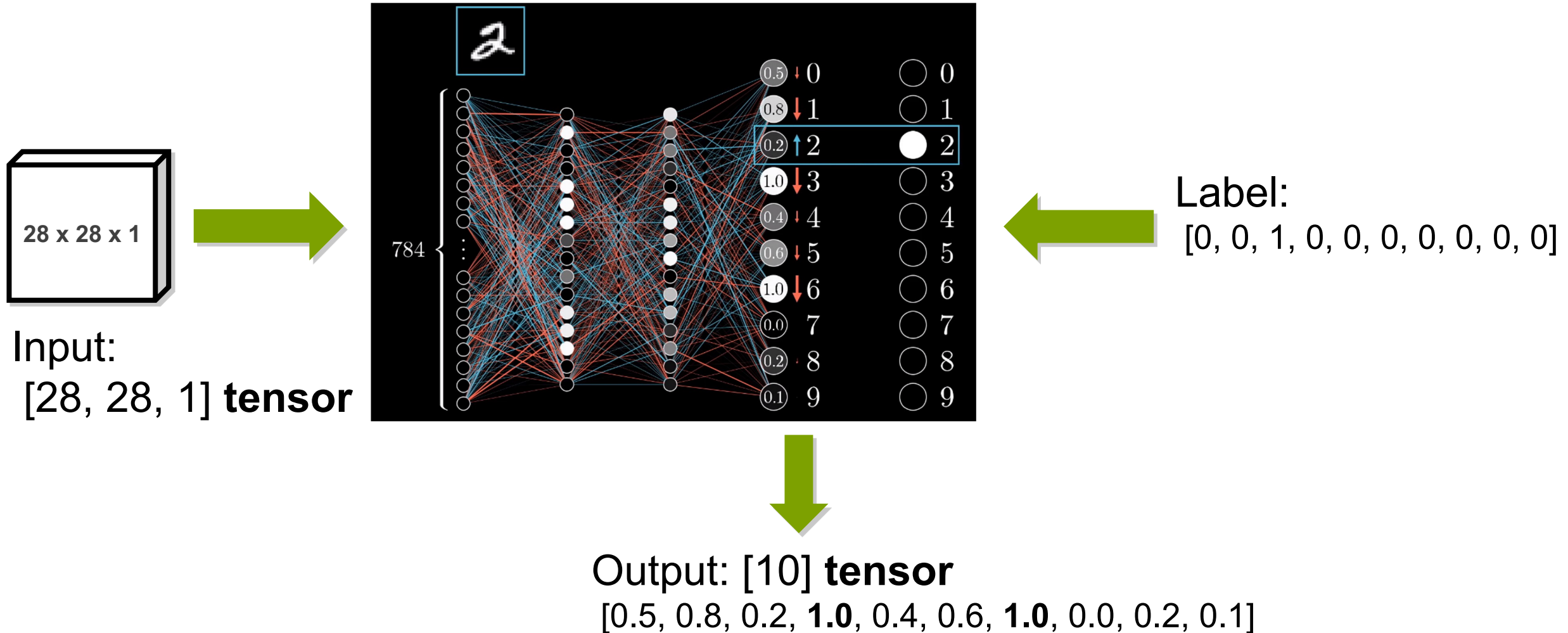
6d-tensor

Machine Learning

- 예시 – Object classification



Machine Learning





1-3

Pytorch basics

Machine Learning Frameworks

- **PyTorch : Facebook이 개발한 머신 러닝 프레임워크**

- Define-by-Run
- Dynamic graph



- **TensorFlow (v1) : Google이 개발한 머신 러닝 프레임워크**

- Define-and-Run
- Static graph

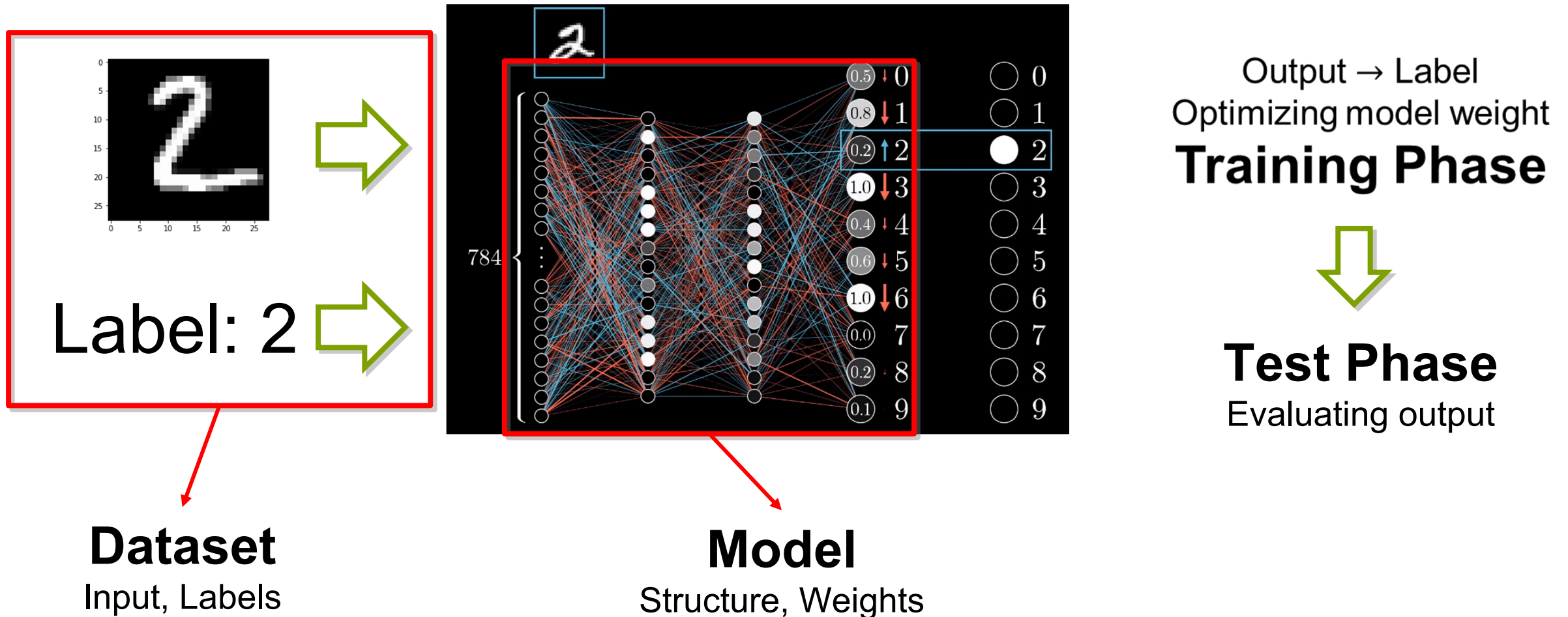
- **TensorFlow (v2) : 2019년 출시**

- PyTorch와 비슷한 방식으로 변경



Pytorch basics

- Pytorch를 활용한 딥러닝 프로그램의 기본 구조



Pytorch basics

- Pytorch를 활용한 딥러닝 프로그램의 기본 구조
 - Model(nn.Module) : Class
 - 전체적인 인공신경망의 구조를 선언하는 부분
 - `__init__`, `forward` 등의 method는 꼭 필요함
 - Dataset : Class
 - 학습에 필요한 데이터셋을 선언하는 부분
 - `__init__`, `__len__`, `__getitem__`의 method는 꼭 필요함
 - Training
 - 학습을 진행하는 부분
 - {Validation & Test}
 - 학습된 결과를 확인하는 부분

Pytorch basics

- **Model**

```
Class Model(nn.Module):  
    def __init__(self, HYPERPARAMETERS):  
        super(Model, self).__init__()  
        ...SOME DECLARATION...  
  
    def forward(self, INPUTS):  
        ...SOME COMPUTATIONS...  
        return OUTPUT
```

Pytorch basics

- Dataset

```
Class Dataset(Dataset):  
    def __init__(self, PARAMETERS):  
        ...SOME DECLAIRATION...  
  
    def __len__(self):  
        return length_of_dataset  
  
    def __getitem__(self, index):  
        return Dataset[index]
```


Pytorch basics

■ Training part

```
DECLAIR DATASET, DATALOADER
```

```
DECLAIR HYPERPARAMETERS
```

```
model = Model(HYPERPARAMETERS)
```

```
optimizer = OPTIMIZER(model.parameters(), lr=LEARNING_RATE)
```

```
for epoch in range(NUM_EPOCHES):
```

```
    for (INPUTS, REAL_VALUES) in DATALOADER:
```

```
        optimizer.zero_grad()
```

```
        OUTPUTS = model(INPUTS)
```

```
        loss = LOSS_FUNCTION(OUTPUT, REAL_VALUES)
```

```
        loss.backward()
```

```
        optimizer.step()
```

Codes

- 실습 코드 Github 주소
 - https://github.com/cws7777/Al_Expert_Lecture_Files